

Stratégie 2 — Suivre la tendance : croisement de SMA et MACD

Première famille de stratégies : le **suivi de tendance** — acheter la force, vendre la faiblesse. On en construit deux implémentations : le **croisement de moyennes mobiles** SMA 50/200 (lent, le classique *golden cross*) et le **MACD** 12/26/9 (rapide, deux moyennes exponentielles). Même camp, même marché, mêmes frais — le face-à-face isole donc **un seul facteur : la réactivité**. Verdict mesuré, contre-intuitif : le lent écrase le rapide.

Les deux stratégies restent **long/flat** (investi à 100 % ou entièrement en cash, jamais short) : c'est ce qui les rend directement comparables au Buy & Hold de la Stratégie 1 — même univers, mêmes frais, seule la *présence sur le marché* change.

Le croisement de moyennes : SMA 50/200

Deux moyennes du prix : une **rapide** (50 h $\approx$ 2 jours) et une **lente** (200 h $\approx$ 8 jours). Quand la rapide passe au-dessus de la lente, la tendance courte devient plus forte que la tendance de fond → on achète ; quand elle repasse en-dessous → on retourne 100 % cash.

Le point technique central : un croisement est un **changement de signe** de rapide - lente, pas un état. « La rapide est au-dessus » peut durer des semaines ; « elle **vient** de passer au-dessus » n'arrive qu'une fois. Il faut donc mémoriser le signe **précédent** — c'est la variable `diff_prec`. Cœur de `sma_croisement.py` (lecteur, frais et `SymbolProperties` copiés tels quels de la Stratégie 1) :

```
PERIODE_RAPIDE = 50          # SMA courte (50 h  $\approx$  2 jours)
PERIODE_LENTE = 200          # SMA longue (200 h  $\approx$  8 jours)

def initialize(self):
    # ... (bornes adaptatives, cash, UTC, SymbolProperties, 24/7, FeeModel :
    #   STRICTEMENT identiques à buyhold.py) ...
    # Deux moyennes mobiles, nourries À LA MAIN dans on_data - transparence
    maximale :
    # on voit exactement avec quelle donnée l'indicateur se met à jour.
    self.sma_rapide = SimpleMovingAverage(PERIODE_RAPIDE)
    self.sma_lente = SimpleMovingAverage(PERIODE_LENTE)
    self.diff_prec = None        # signe précédent de (rapide - lente)

def on_data(self, data: Slice):
    if self.btc not in data:
        return
    close = float(data[self.btc].value)

    # 1) Nourrir les indicateurs avec le close de la barre qui vient de
    CLÔTURER
    self.sma_rapide.update(data[self.btc].end_time, close)
    self.sma_lente.update(data[self.btc].end_time, close)
    if not (self.sma_rapide.is_ready and self.sma_lente.is_ready):
        return    # warmup : pas encore 200 barres -> aucun signal

    # 2) Détecter le croisement (changement de signe de rapide - lente)
    diff = self.sma_rapide.current.value - self.sma_lente.current.value
    if self.diff_prec is not None:
        croise_haut = self.diff_prec <= 0 and diff > 0    # rapide passe AU-
        DESSUS
        croise_bas = self.diff_prec >= 0 and diff < 0    # rapide passe EN-
        DESSOUS
        if croise_haut and not self.portfolio[self.btc].invested:
```

```

        self.set_holdings(self.btc, 1.0)                # entrée : tout en BTC
    elif croise_bas and self.portfolio[self.btc].invested:
        self.liquidate(self.btc)                        # sortie : 100 % cash
    self.diff_prec = diff

```

Le journal des trades vit dans `on_order_event` (compteur, frais cumulés, sens ACHAT/VENTE) et le bilan dans `on_end_of_algorithm` — même mécanique qu'en Stratégie 1.

```

dotnet QuantConnect.Lean.Launcher.dll `
--algorithm-language Python `
--algorithm-type-name SmaCroisement `
--algorithm-location "../..../algorithms/sma_croisement.py"

```

Résultat mesuré (extraits — tes chiffres évolueront avec le CSV, la structure est la même) :

```

TRADE  1 ACHAT 2026-01-13 19:00 UTC | qté=+1.06549619 @ 93581.0 | frais=39.88 $
TRADE  2 VENTE 2026-01-19 22:00 UTC | qté=-1.06549619 @ 92968.5 | frais=39.62 $
...
--- BILAN Croisement SMA 50/200 ---
Trades exécutés : 22 | frais totaux : 826.64 $
Équité finale : 95723.48 $ | rendement stratégie : -4.2765%
Buy & Hold (close/close) : -33.2310% | écart stratégie - B&H : +28.9545%

```

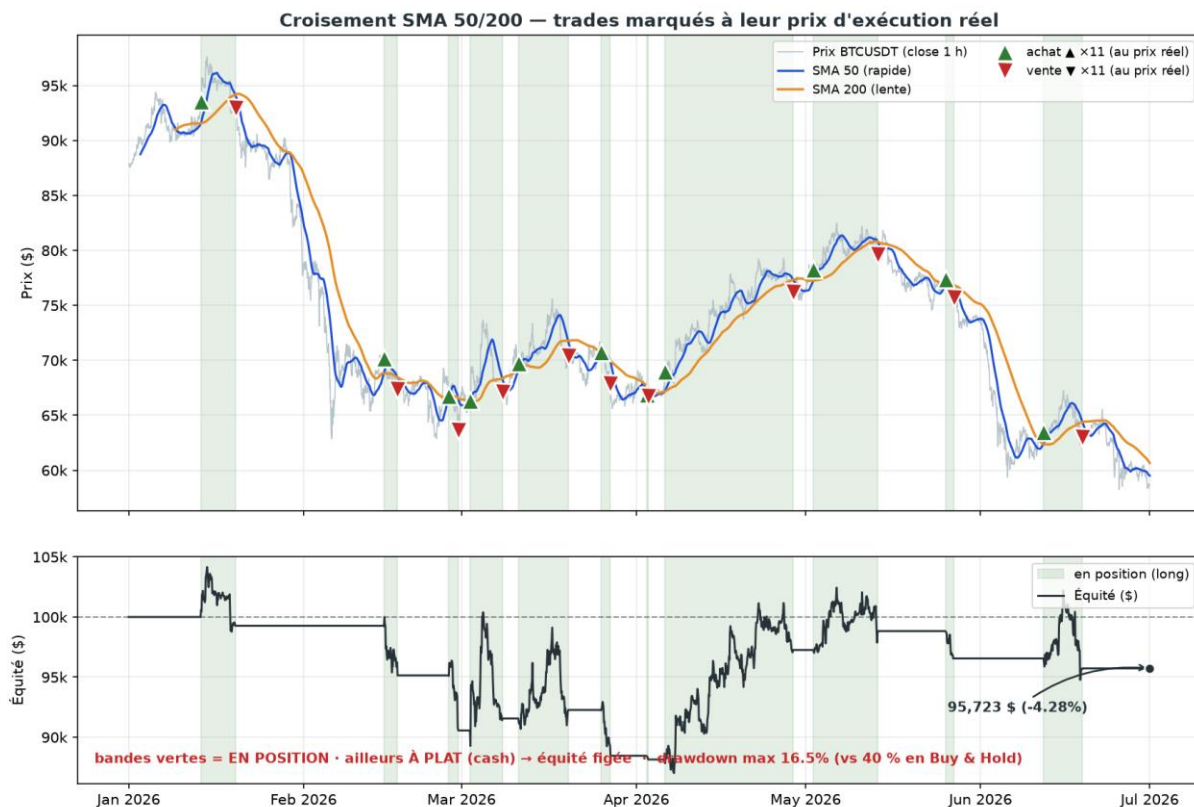
STATISTICS:: Net Profit -4.277% STATISTICS:: Drawdown 16.500%

Trois détails de lecture : le premier trade n'a lieu que le **13 janvier** (les 200 premières barres sont le warmup) ; 22 ordres = 11 aller-retours long/flat ; et la quantité change à chaque entrée (`set_holdings(1.0)` redimensionne selon le prix et l'équité du moment).

Le face-à-face avec Buy & Hold

	Buy & Hold	Croisement SMA 50/200
Ordres	1	22
Frais totaux	39,88 \$	826,64 \$
Net Profit	-33,18 %	-4,28 %
Drawdown max	40,2 %	16,5 %

1. **La stratégie bat nettement le Buy & Hold (+29 points) — en perdant moins, pas en gagnant.**
Sur un marché qui chute de 33 %, rester à plat pendant les pires jambes de baisse limite la casse à -4 %. Le suivi de tendance **coupe les pertes**.
2. **Le drawdown est divisé par ~2,5** (16,5 % vs 40 %) — pour un investisseur réel, souvent plus important que le rendement : c'est la douleur qui fait abandonner une stratégie.
3. **Mais elle perd quand même, et les frais pèsent** : 827 \$ = 0,8 point de rendement. Un tiers des aller-retours sont des *whipsaws* (entrer, ressortir quelques heures plus tard) — le TRADE 13→14 dure 5 heures et coûte ~309 \$ net.



Deux panneaux alignés sur le même axe temporel. En haut : prix horaire BTCUSD avec SMA 50 et SMA 200 ; chaque trade est marqué à son prix d'exécution réel (triangle vert = achat, rouge = vente), les périodes de détention sont ombrées en vert. En bas : équité avec les mêmes bandes vertes ; hors des bandes, l'équité est figée car la stratégie est à plat (en cash).

Figure — Chaque bande verte est un aller-retour : achat ▲ à son bord gauche, vente ▼ à droite, au prix réel. Hors des bandes on est à plat (sans position — le marché, lui, cote toujours) : l'équité est figée.
(Script : fig_sma_croisement.py)

Un mot d'honnêteté d'exécution : le fill du TRADE 1 se fait à **93 581,0** — le close exact de la barre du croisement. **On décide et on exécute au même prix** : c'est la convention optimiste mesurée en Moteur 2 (fill immédiat au dernier prix connu, et en 24/7 le marché est toujours ouvert). Ce n'est pas un regard vers le futur (on ne décide que sur des barres clôturées), mais c'est une hypothèse favorable, assumée — et qui devra être **identique** côté vectorbt pour que les moteurs tombent d'accord.

Le MACD : plus réactif, donc mieux ?

Le **MACD** (*Moving Average Convergence Divergence*) empile trois lectures d'une même tendance : la **ligne MACD** = $EMA(12) - EMA(26)$ (l'EMA pèse davantage les barres récentes qu'une SMA — plus réactive) ; la **ligne de signal** = $EMA(9)$ de la ligne MACD ; et l'**histogramme** = $MACD - \text{signal}$, qui **change de signe exactement au croisement** des deux lignes. Règle de trading : achat quand le MACD passe au-dessus de son signal, vente au croisement inverse — le jumeau du diff_prec, appliqué à l'histogramme. Cœur de macd.py :

RAPIDE, LENTE, SIGNAL = 12, 26, 9

réglages classiques du MACD

```

def initialize(self):
    # ... (identique aux autres stratégies) ...
    self.macd = MovingAverageConvergenceDivergence(RAPIDE, LENTE, SIGNAL,
MovingAverageType.EXPONENTIAL)
    self.hist_prec = None

def on_data(self, data: Slice):
    if self.btc not in data:
        return
    close = float(data[self.btc].value)
    self.macd.update(data[self.btc].end_time, close)
    if not self.macd.is_ready:
        return

    hist = float(self.macd.current.value) -
float(self.macd.signal.current.value)
    if self.hist_prec is not None:
        croise_haut = self.hist_prec <= 0 and hist > 0    # MACD passe au-dessus
du signal
        croise_bas = self.hist_prec >= 0 and hist < 0
        if croise_haut and not self.portfolio[self.btc].invested:
            self.set_holdings(self.btc, 1.0)
        elif croise_bas and self.portfolio[self.btc].invested:
            self.liquidate(self.btc)
    self.hist_prec = hist
dotnet QuantConnect.Lean.Launcher.dll `
--algorithm-language Python `
--algorithm-type-name Macd `
--algorithm-location "../../../../../algorithms/macd.py"

```

Résultat mesuré :

```

--- BILAN MACD (12,26,9) ---
Trades exécutés : 341 | frais totaux : 11692.11 $
Équité finale : 67850.76 $ | rendement stratégie : -32.1492%

```

STATISTICS:: Net Profit -32.149% STATISTICS:: Drawdown 33.100%
Le chiffre qui doit faire sursauter : **341 ordres**, et son corollaire **11 692 \$ de frais** — **~12 % du capital de départ** envolés en commissions. Quinze fois plus de trades que la SMA.

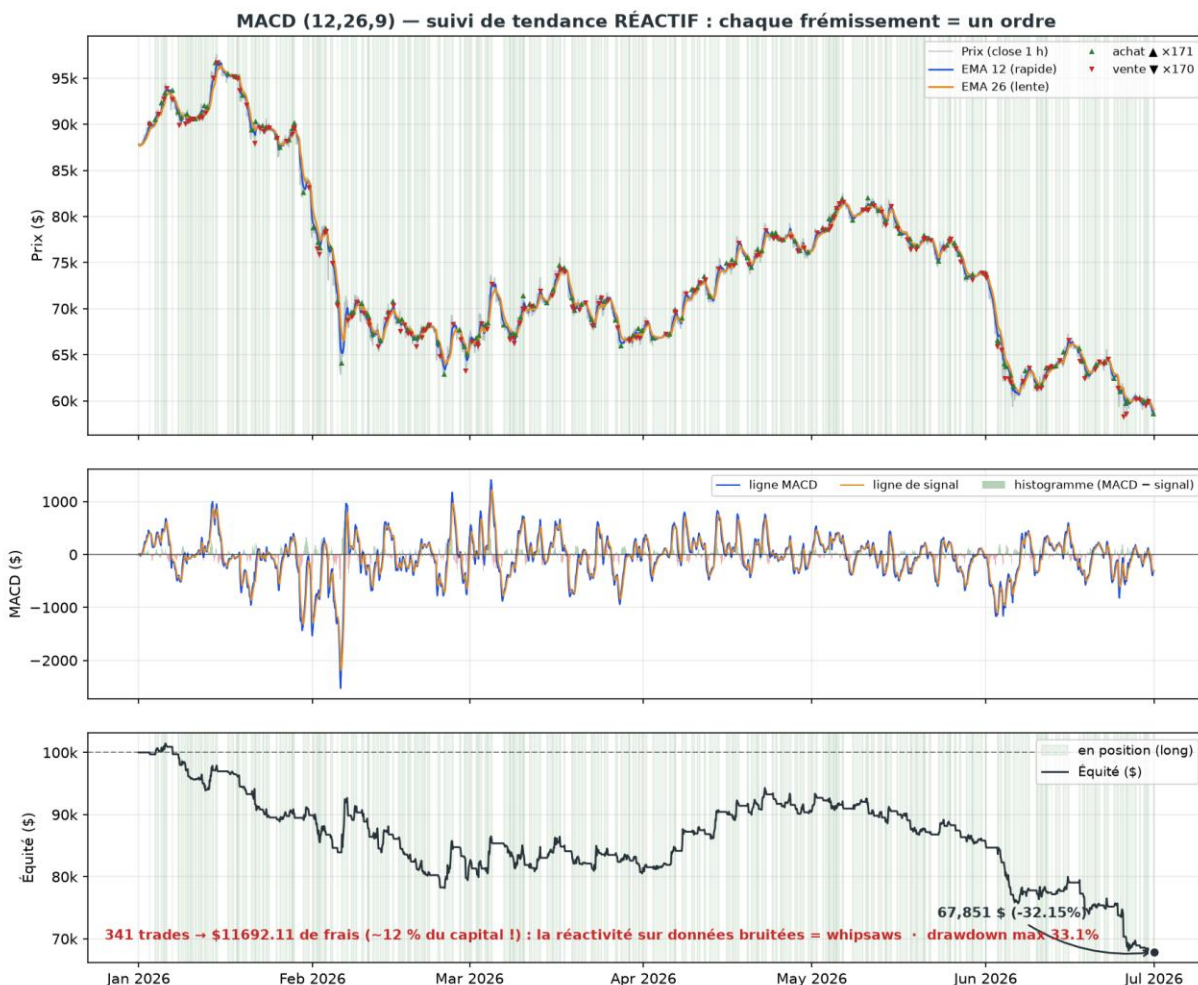
Lent contre rapide : le verdict

	SMA 50/200 — lent	MACD 12/26/9 — rapide
Net Profit	-4,3 %	-32,1 %
Drawdown	16,5 %	33,1 %
Trades	22	341
Frais	827 \$	11 692 \$
Sharpe	-0,46	-1,64

Même camp — suivre la tendance — et des résultats aux antipodes. Explication : le prix horaire est **bruité**. La SMA 200 est si lente qu'elle ignore ce bruit (22 croisements, les vrais retournements). Le MACD 12/26 réagit à chaque frémissement : 341 croisements, presque tous des whipsaws — on entre,

le bruit s'inverse, on sort, on paie deux fois les frais. **La réactivité, précieuse pour capter tôt une tendance, devient un défaut quand il n'y a pas de tendance à capter.**

La leçon profonde : **12/26/9 sont des réglages pensés pour le graphe journalier.** Sur du horaire, « 12 heures vs 26 heures » est une fenêtre minuscule, dominée par le bruit. Un indicateur n'a de sens **qu'accordé à son échelle de temps.**



Trois panneaux alignés : prix avec EMA 12/26 recouvert d'un mur dense de triangles d'achat et de vente ; ligne MACD et signal quasi collées qui se croisent sans cesse autour de zéro ; équité qui glisse jusqu'à -32,1 %.

*Figure — Le panneau du milieu dit tout : MACD et signal se croisent en permanence ; chaque croisement = un trade. Sur données horaires, ce momentum mesure du **bruit**. (Script : fig_macd.py)*

Dernier point, qui prépare le capstone : le MACD offre **deux lectures** — le **croisement** MACD/signal (le *timing*) et le **signe** de la ligne MACD (le *régime* : sommes-nous en tendance haussière ?). Notre stratégie n'utilise que le timing, et achète donc même en pleine tendance baissière — c'est là que naissent les whipsaws les plus chers. Exiger les deux lectures **ensemble** (croisement ET MACD > 0) filtre les entrées à contre-tendance et divise les frais. La Stratégie 6 (stratégie avancée) poussera cette idée jusqu'au bout : un filtre de régime + un signal de timing + une gestion du risque.

Ce qu'il faut retenir

- **Un croisement = un changement de signe**, pas un état : mémoriser le pas précédent (`diff_prec`, `hist_prec`) est indispensable — tester l'état ferait trader chaque barre.
- **Warmup** : aucun signal tant que `is_ready` est faux (200 barres pour la SMA lente) ; le premier trade « en retard » est normal et voulu.
- **En marché baissier, le suivi de tendance coupe les pertes** (SMA : -4,3 % et drawdown 16,5 % contre -33,2 % et 40 % en B&H) — il limite la casse, il ne crée pas de gain.
- **Rapide n'est pas mieux** : le MACD, plus riche et plus réactif, fait quinze fois plus de trades et douze fois plus de frais pour un résultat bien pire. Les paramètres se pensent **par timeframe**, et les frais croissent avec la fréquence — c'est désormais *le* problème central de conception.

Précédent : [Stratégie 1 — Buy & Hold](#) · Suite : [Stratégie 3 — Le retour à la moyenne : RSI et Bollinger](#), le camp opposé — acheter la faiblesse — et son trade catastrophe.